2023 5th International Conference on Sustainable Technologies for
Industry 5.0 (STI), 09-10 December, Dhaka

A Machine Learning Approach for Predicting Efficient CPU Scheduling Algorithm

Swapnil Biswas, Md. Shakil Ahmed, Md. Jobayer Rahman,
Anika Khaer and Md. Motaharul Islam
Department of Computer Science and Engineering, United International University, Bangladesh
Email: sbiswas192072@bscse.uiu.ac.bd, mahmed192077@bscse.uiu.ac.bd, mrahman192070@bscse.uiu.ac.bd,
akhaer192082@bscse.uiu.ac.bd, motaharul@cse.uiu.ac.bd

***Abstract*—CPU scheduling is a critical aspect of an operating system design as it dictates how processes are prioritized for execution. In a uniprocessor, only a single process at a time can be executed simultaneously. The CPU scheduler considers various factors to determine the most effective process order. Thus, it's vital to decide quickly on the CPU scheduling strategy. In this research, a number of different machine learning algorithms are compared in order to determine which one offers the most suitable CPU scheduling method that is very effective and has the quickest turnaround time to carry out the new process requests. Implementing machine learning techniques in operating systems research can improve critical aspects such as response time, throughput, turnaround time, and waiting time. By enhancing the overall performance of the system, users are delivered a faster and more efficient computing experience. This analysis showcases the significance of utilizing machine learning techniques as a solution to the issues arising due to the ever-increasing complexity and demands of modern computing. By optimizing resource utilization and improving overall performance, we can reduce energy consumption and environmental impact while meeting the demands of modern computing. This aligns with the objectives of sustainable technology and Industry 5.0 to integrate physical and digital systems for sustainable and efficient production.**

***Index Terms*—Operating System, CPU scheduling, Machine learning model, Turnaround time, Sustainability, Industry 5.0**

I. Introduction

In computer science, scheduling is the process of allocating system resources such as processor time, memory, and bandwidth to various processes, threads, data flows, and applications that require them. A scheduling algorithm is needed because computers need to be fast to handle multitasking and multiplexing. Scheduling is a fundamental component of an operating system that determines which process to run when there are multiple options also involves allocating the processor time to various processes running concurrently on the system. Scheduling the CPU is important because it affects how well resources are used and how well other performance parameters are met. There are a number of CPU scheduling algorithms such as first come first serve [1], shortest job first [2], round robin [3], and priority scheduling [4], but each one works best in different situations in real-time operating systems.

A process is a program that is already in execution, while those waiting to be processed are placed in a queue commonly known as the ready queue shown in figure 1. To optimize the utilization of the CPU, it is crucial to keep it active as much as possible [5]. The short-term scheduler, which is an integral component of the operating system, identifies a waiting a waiting procedure that pulls resources from the ready queue and allocates the CPU whenever it becomes available [6]. CPU scheduling's primary objective is to maximize system performance by ensuring the efficient allocation of resources and minimizing response time.

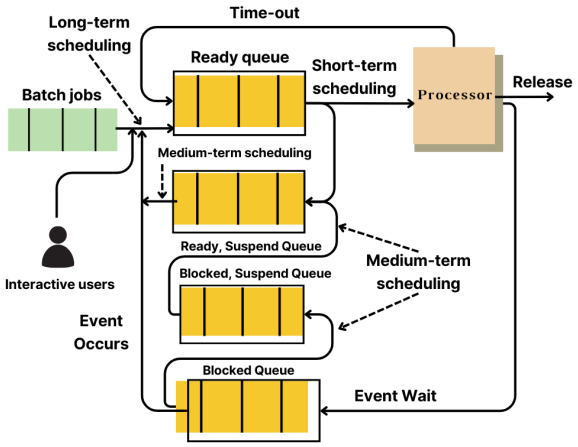

Fig. 1: CPU Scheduling Process

Real-time operating systems are designed to ensure time-critical tasks are completed within their deadlines. These systems are capable of managing large numbers of processes that require different scheduling algorithms to achieve optimal performance, making traditional optimization approaches difficult to implement. Balancing performance and resource usage is a top priority for real-time systems that integrate into a larger computing infrastructure. Cutting-edge technologies, such as machine learning and real-time analytics, help real-time systems to analyze and learn about the behavior of processes and enable developers to identify potential bottlenecks. Integration with cloud and edge computing systems is leading to even greater complexities in real-time computing, with more focus on optimizing the performance rates while also reducing resource consumption for reliable and secure operations. Real-time operating systems are vital for modern computing systems, and their evolution will continue to incorporate new

979-8-3503-9431-3/23/$31.00 © 2023 IEEE

technologies and techniques to handle higher levels of complexity in the future. As a result, we have worked to improve the infrastructure and workflow limitations of CPU scheduling. The major contributions of this paper are as follows:

- Introducing a novel method aimed at resolving the challenge associated with CPU scheduling within the operating system.
- Conducting an in-depth analysis to identify the optimal machine learning models for forecasting efficient scheduling algorithms, resulting in an impressive accuracy rate of 94.56%.
- Enhancing the speed of the scheduling process and elevating overall efficiency.

In section II, III, IV and V represent related works, proposed model, experimental result, and conclusion respectively.

II. Related works

The CPU scheduling problem is a fundamental issue in operating systems and computer technology, which has gained significant attention from researchers and practitioners alike. The development of efficient CPU scheduling algorithms has been a critical area of research for several decades, with many different strategies proposed to achieve optimal performance [7]. Lately, methods based on machine learning have come up as promising ways for predicting efficient CPU scheduling algorithms by analyzing system parameters such as process behavior, resource utilization, and performance metrics [8]–[10]. We explore research studies that have approaches to predicting efficient CPU scheduling algorithms and examine their effectiveness in optimizing system performance. Murad et al. [11] conducted a comparison of the machine learning algorithm and the performance matrices of the priority algorithm. This study includes waiting times, CPU times, turnaround times, and execution start times. The cloudlet is allocated to the CPU for time-sharing and space-sharing modes and VMs are always allotted in space-sharing mode. Pérez et al. [12] proposed a library for evenly distributing a single kernel workload across different devices. Their library utilizes several methods for spreading out the load, including static assignment based on computing power, dynamic allocation of equal-sized work packages to free devices, and adaptive allocation based on a guided approach However, Maat's technique is not suitable for multiple OpenCL [13] workloads mapped on different devices executing concurrently.

In this paper [14], a hybrid scheduling approach is presented that schedules a job in a load-balanced manner on a heterogeneous CPU and GPU system. In the two-phase scheduling method, the original kernel is transformed into the first (i.e., static phase) partition-ready OpenCL kernels designed for the objective of mapping computations onto diverse heterogeneous computing systems. Following the observation of the execution duration of partition-ready kernels, the granularity of partitioned kernels is adjusted during the dynamic phase in order to attain load balancing for iterative kernel execution. Lin et al. [15] introduce a reinforcement learning-based scheduling algorithm for heterogeneous computing environments. It employs a convolutional neural network with graph attention to process scheduling information and a fully connected neural network to select tasks. Experiments demonstrate a 13.03% reduction in average application termination time compared to existing algorithms. Cheng et al. [16] proposed a Deep Reinforcement Learning-based job scheduler that dispatches the jobs in real time to tackle this problem. The primary objective is to prioritize user requests in a manner that ensures high-quality service to end-users, while also achieving substantial cost savings in the execution of tasks on virtual instances. The authors have employed the Deep Q-learning Network model to accomplish their solution.

Multi-access Edge Computing (MEC) systems require efficient scheduling and offloading of tasks onto servers to function properly which requires effective resource allocation (RA) mechanisms. Machine Learning (ML) and Deep Learning (DL) are useful tools in addressing RA challenges in MEC. This study [6] explores ML/DL-based RA mechanisms in MEC by reviewing recent research that used these techniques. Bayesian decision theory was suggested by Naila et al. as a way to sort the processes in the ready queue into groups [17]. They put the processes into useful and not-useful groups based on the data's static and dynamic properties. This selection process consists of two phases; in the first phase, it compares the static and dynamic properties of the new processes in the queue with the properties of the datasets of the previously executed processes. Bal et al. [18] utilized an improved feline multitude advancement calculation, optimized clustering-based deep neural network, and a lightweight authentication strategy in their new machine learning algorithm for task scheduling and resource allocation in cloud computing environments. Their proposed RATS-HM method was compared to existing CPU usage and response time systems like ITSEPM, FCFS, and Round Robin systems. The issue of scheduling the allocation of diverse dispersed resources in cloud computing has garnered significant interest among researchers in the field. On the other hand, Vasiliadis et al. [19] employ a data-driven methodology and provide an adaptive scheduling strategy that is designed for machine learning inference tasks and permits the utilization of the most effective processing equipment available.

Helmy et al. came up with a way to estimate CPU burst time in computational grids that are based on machine learning [20]. These authors used 16 characteristics of the processes as "feature sets" and then used the K Nearest Neighbor, Support Vector Machine (SVM), Artificial Neural Network (ANN), and Decision Tree methods to figure out the burst time. With these algorithms, they got about 94% to 97% accuracy. This study shows that the CPU burst time and process attributes are linked in a straight line. Gao et al. [21] measured and compared the state-of-the-art statistical and machine learning methods in task workload prediction using the Google cluster trace. They suggested the m-gap prediction method to estimate the workload a certain amount of time before the predicted time point. This would give enough time to schedule tasks based on the estimated workload. Also proposed a clustering-based

workload prediction method for higher prediction accuracy. Our proposed algorithm is designed to address the challenges faced by traditional CPU scheduling algorithms by leveraging the power of machine learning. The results of our research have the potential to revolutionize CPU scheduling and improve system performance and efficiency.

III. Proposed Model

In this research, we explored a range of models to address the optimal solutions. Following that, we present experimental results that provide insights into the performance of our models.

A. Proposed system Workflow

The implementation of our proposed model, as depicted in Figure 2, occurs within the job queue. When a process is introduced into the system, it is placed in the Job queue. Initially, a novel procedure is placed into the ready queue. The process remains in the ready queue until it is selected for execution. Then, a machine learning algorithm will decide which CPU scheduling algorithm is best for this process. The process is then given to the CPU and begins to run. One of the following things can happen:

- The process may send a request for input/output and then be placed in a queue for input/output.
- The primary process might initiate a new one and then wait for it to terminate.
- An interrupt may forcibly remove the process from the CPU and place it back in the ready queue.

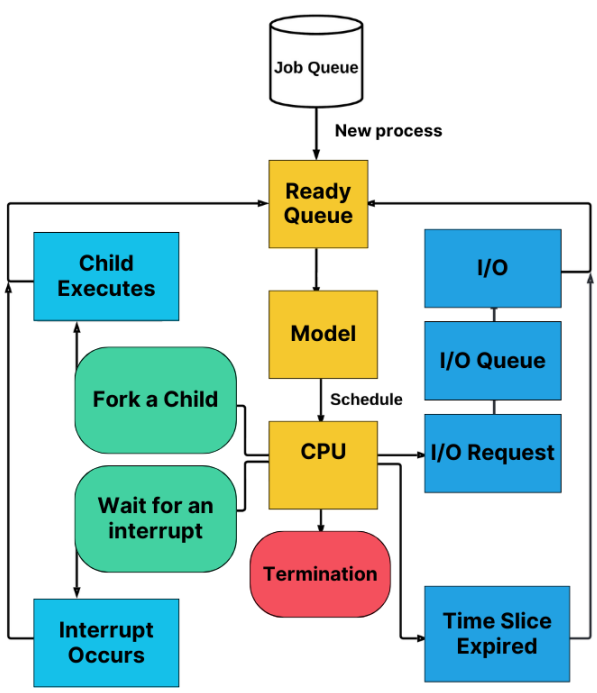

Fig. 2: Workflow of Proposed Procedure

The process transitions from the waiting state to the ready state and then back to the ready queue in the first two cases. This cycle persists until the task is finished. At that point, it is removed from all queues and its process control block, which is a type of data structure used to store information about a process that may be needed to control its scheduling, and its resources are given to another process. From the ready queue, comes the model section.

B. Proposed System Diagram

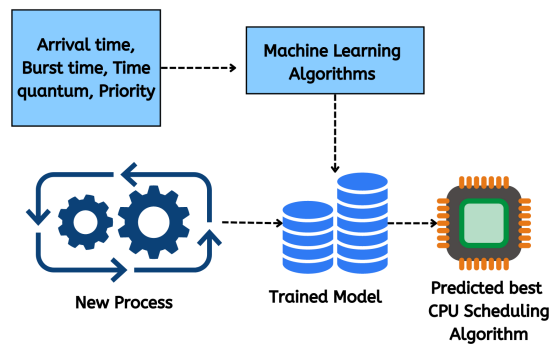

Fig. 3: Proposed System Diagram

Our algorithm, as shown in figure 3, gives priority to the startup duration of a process and selects the one requiring the least amount of time for execution. If a new process is added while a process is running, it is queued for later processing. In case the queue is empty, the first unfinished process in the queue is executed. However, if the queue is full, a request is generated. Subsequently, three machine learning algorithms are employed to analyze factors such as previous arrival time, burst time, quantum, and priority to determine the most optimal CPU scheduling algorithm for processing the new requests. The chosen scheduling algorithm is the one with the fastest turnaround time and greatest efficiency for running the new process.

C. Model Explored

*1) **Stochastic Gradient Descent (SGD):*** Gradient Descent is the most common optimization strategy used in machine learning frameworks [22]. Gradient Descent, to put it simply, iterates over a function, modifying its parameters until it locates the minimum. By modifying weights after going through each sample in the training set, Gradient Descent is utilized to minimize error. A stochastic gradient descent occurs when the weights are updated after a predetermined subset of training samples or after each sample in the training set. In contrast, SGD updates a parameter for every training example $\theta^{(t)}$ and labels y_i in equation 1. SGD eliminates this redundancy by carrying out updates one at a time. As a result, it is typically significantly quicker and can be used for online learning. Due to SGD's frequent updates and large variation, the goal function is highly erratic.

$$\theta = \theta - \eta.\nabla_\theta J(\theta; x^{(i)}; y^{(i)}) \tag{1}$$

$\theta^{(t)} =$ *is the current estimate of* θ^*

$\eta =$ *the learning rate*

$x^{(i)} =$ *training example (burst time, arrival time, priority, time quantum)*

$y^{(i)} =$ *label(FCFS, SJF, SRTF, PSA, RR)*

$\nabla_\theta J(\theta; x^{(i)}; y^{(i)}) =$ *the gradient of the loss function*

The following are the main steps for deploying our model using multiple stochastic gradient descents:

1) Initialize all the values of x (burst time, arrival time, priority, time quantum) and y (FCFS, SJF, SRTF, PSA, RR).
2) Calculate the Mean Squared Error (MSE) for the provided dataset and subsequently update the value of 'n' in a sequential manner, first by calculating it separately for 0 and 1.
3) For a constant number of epochs, we will run the algorithm for an equal number of iterations.

SGD is an efficient algorithm that looks at a different training set with each iteration, resulting in faster parameter changes compared to regular gradient descent. The number of iterations increases with the number of datasets, but this approach sacrifices a small amount of accuracy for speed. SGD requires more rounds to reach the minimum point because of its random descent behavior, but it is still less computationally exhaustive than Batch Gradient Descent.

2) ***Logistic Regression****:* A statistical method for simulating the relationship between one or more independent variables and a dependent variable is regression analysis. It is frequently used to find the correlation between variables and for predictive modeling. It is commonly used for predictive modeling and to determine the correlation between variables. Overfitting can also happen with logistic regression [23], especially when there are a lot of predictor variables in the model. When a model has a lot of dimensions, regularization is often used to punish parameters with large coefficients. It indicates the significant relationships between the dependent variable and the independent variable. It indicates the strength of the impact of multiple independent variables on a dependent variable.

$$\ln[\frac{\hat{p}}{1-\hat{p}}] = b_0 + b_1X_1 + b_2X_2 + ... + b_pX_p \tag{2}$$

$\hat{p} =$ *dependent variables (FCFS, SJF, SRTF, PSA, RR)*

$X_1...X_p =$ *independent variables (arrival time, burst time, priority, time quantum)*

$b_0...b_p =$ *the regression coefficients*

Here, $\hat{p}$ indicates the predicted probability that the event will occur, X_1 through X_p are separate independent variables (arrival time, burst time, priority, and time quantum), and b_0 through b_p are the regression coefficients that describe the relationship between the variables in equation 2.

The main steps for implementing our model using logistic regression are as follows:

- We split the data into dependent and independent variables. We consider the CPU scheduling algorithms as the dependent variable and arrival time, burst time, priority, and time quantum as the independent variables.
- The data set was divided into a training set and a test set. We train our logistic regression algorithm using the training set. The logistic regression model will also be validated using the test data set.
- To improve the accuracy of our model, we rescale the data to align values.

The logistic regression model generates probabilities ranging from 0 to 1. To make predictions about the training data (y train pred), we transform these probabilities into 1s and 0s by choosing a probability threshold value. We compare the predicted values (y train pred) to the actual values (y) and generate predictions for the Converted variable values by forecasting either 0 or 1.

3) ***Support Vector Machine****:* Support Vector Machine (SVM) [24] is a supervised and linear Machine Learning algorithm most commonly used for solving classification problems. Depending on the kernel function selection, SVM performs sophisticated data transformations. It attempts to optimize the separation boundaries between your data points according to the labels or classes chosen. SVM searches for a line that optimizes the distance between two classes of 2-dimensional space points in its simplest configuration, known as linear separation. The objective of generalization is to identify a hyperplane within an n-dimensional space that maximizes the distance between data points and their respective classes. Support vectors refer to the specific data points that exhibit the closest proximity to the hyperplane, while also being removed from it by the minimum distance. The optimization challenge that SVMs attempt to solve is expressed in the following formula 3 and 4:

$$min_{\omega,b,\zeta}\frac{1}{2}w^Tw + C\sum_{i=1}^{n}\zeta_i \tag{3}$$

$$y_i(w^t\phi(x_i) + b) \geq 1 - \zeta_i \tag{4}$$

$\zeta_i =$ *denotes the distance between FCFS, SJF, SRTF, PSA, and RR).* $\zeta_i \geq 0, i = 1, ..., n$

$C =$ *denotes a regularization parameter*

$w^tw = ||w^2||$*denotes the normal vector*

$\phi(x_i) =$ *denotes the transformed input (arrival time, burst time, priority, time quantum) space vector*

$b =$ *denotes a bias parameter*

$y_i =$ *denotes the i-th target value*

The following are the fundamental steps for applying our model using Support Vector Machine:

1) We predict the classes; in our case, the CPU scheduling algorithms. One of the classes is identified as 1, while the other is identified as -1.
2) We use a loss function to find the maximum margin, then the error/loss is calculated.
3) In our case, weights are optimized by calculating the gradients using advanced mathematical concepts from calculus.
4) We update the gradients only by using the regularization parameter when there is no error in the classification, while the loss function is also used when misclassification happens.

After fitting the model, we can utilize it to forecast which CPU scheduling algorithm will be the most efficient for the test group. We can compare the model predictions with the actual observations in the test data. To evaluate how effectively the SVM predicts heart disease in the test set, we will calculate its accuracy. The accuracy is measured by the proportion of accurately predicted observations.

IV. Experimental Result

In our proposed model to find the best scheduling algorithm, we have used Stochastic Gradient Descent (SGD), Logistic Regression (LR), and Support Vector Machine (SVM) because they're fast and deliver accurate results in our tests. These models fit well for real-time scheduling. While we explored others, these three were the best in terms of efficiency and performance.

1) Stochastic Gradient Descent(SGD): We experimented with $x = 0.01$ for 2000 iterations and contrasted it with $x = 0.001$ to see which had a higher learning rate in figure 4. Furthermore, we discovered that 0.01 is the more efficient learning rate because it converges considerably more quickly than 0.001. While 0.001 requires 1000 worth of iterations to converge, 0.01 does so around the 100 level.

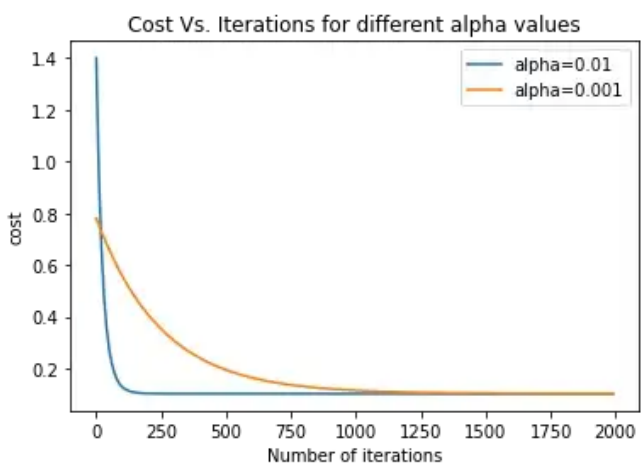

Fig. 4: Cost Vs. Iteration graph for Stochastic Gradient Descent

2) Logistic Regression(LR): The true positive rate and false positive rate are plotted on this ROC (Receiver Operating Characteristic) curve in figure 5. It demonstrates how sensitivity and specificity can be compromised. The true positive rate is the proportion of instances that have been correctly classified, whereas the false positive rate is the proportion of instances that have been incorrectly categorized. The ROC curve's vertical axis represents the genuine positive rate. The ROC curve hugs the top left corner of the graph more closely the better the model performs in categorizing the data. We estimate the AUC (area under the curve), which indicates how much of the plot is underneath the curve, in order to put this into numerical perspective.

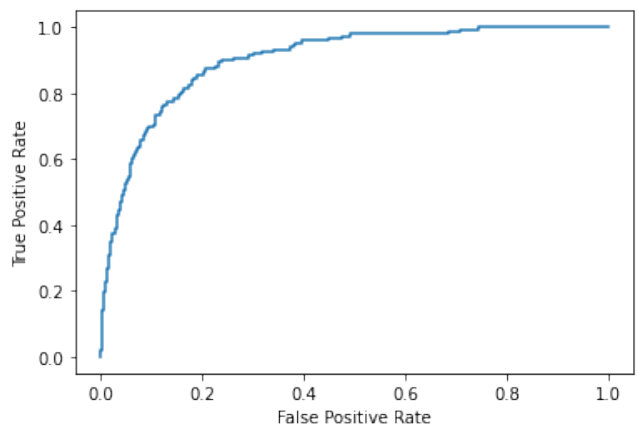

Fig. 5: ROC curve for Logistic regression

3) Support Vector Machine(SVM): This Confusion matrix in figure 6 gives us a clear picture of model performance and the types of errors produced by the model. It gives us a summary of correct and incorrect predictions broken down by each category. The confusion matrix shows $1096 + 77 = 1173$ correct predictions and $15 + 6 = 21$ incorrect predictions

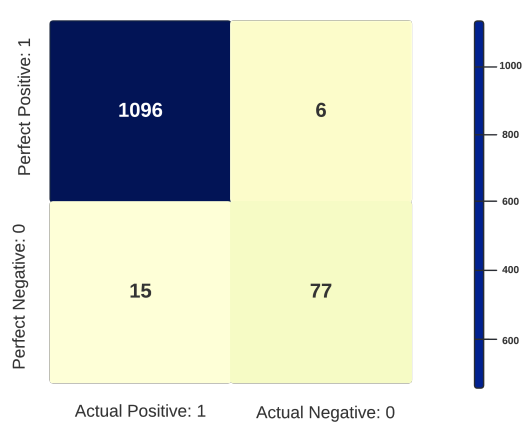

Fig. 6: Confusion matrix of Support Vector Machine

TABLE I: Performance Comparison of Different Models

Model Name	Accuracy(%)	Mean Absolute Error	Root Mean Sequence Error
SVM	94.56	4.18	5.92
LR	90.19	7.0	8.46
SGD	87.26	8.73	8.90

The above experiment in table I shows that by choosing the most efficient CPU scheduling strategy, overall waiting time and average turnaround time may both be decreased. The decrease in overall waiting and turnaround times demonstrates optimal CPU usage and quick responses. Among the three algorithms that we have explored, the support vector machine is the most effective algorithm, despite the fact that the other two algorithms are also helpful. In addition, the performance of the logistic regression algorithm will be improved if the number of samples used is increased. This is because the algorithm will be able to discover more relationships between the variables while it is being trained.

V. Conclusion

Choosing the right CPU scheduling algorithm is crucial, as it directly affects CPU performance and workflow mapping. Support vector machines were found to be superior to stochastic gradient descent and logistic regression methods for making this choice, with faster processing times and improved performance. This approach can be expanded upon in the future by choosing a scheduling algorithm for different queues and file types simultaneously, leading to higher expected output and greater accuracy. However, there are some limitations to this approach, including the need to further investigate the performance and accuracy of the proposed model for other types of data and scheduling algorithms. Future work may also involve improving the scalability and reliability of time-sharing systems using emerging technologies like cloud computing and virtualization. Strategies for improving the distribution of computational resources, managing the system's workload, and improving the fault tolerance of the system could be explored.

References

[1] S. Ali, R. Alshahrani, A. Hadadi, T. Alghamdi, F. Almuhsin, and E. Sharawy, "A review on the cpu scheduling algorithms: Comparative study," *Int. J. Comput. Sci. Netw. Secur*, vol. 21, no. 1, pp. 19–26, 2021.

[2] M. Hamayun and H. Khurshid, "An optimized shortest job first scheduling algorithm for cpu scheduling," *J. Appl. Environ. Biol. Sci*, vol. 5, no. 12, pp. 42–46, 2015.

[3] R. Mishra and G. Mitawa, "Improved round robin algorithm for effective scheduling process for cpu," in *2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV)*. IEEE, 2021, pp. 590–593.

[4] S. Simaiya, V. Gautam, U. K. Lilhore, A. Garg, P. Ghosh, N. K. Trivedi, and A. Anand, "Eepsa: Energy efficiency priority scheduling algorithm for cloud computing," in *2021 2nd International Conference on Smart Electronics and Communication (ICOSEC)*. IEEE, 2021, pp. 1064–1069.

[5] A. Grammenos, T. Charalambous, and E. Kalyvianaki, "Cpu scheduling in data centers using asynchronous finite-time distributed coordination mechanisms," *IEEE Transactions on Network Science and Engineering*, 2023.

[6] H. Djigal, J. Xu, L. Liu, and Y. Zhang, "Machine and deep learning for resource allocation in multi-access edge computing: A survey," *IEEE Communications Surveys & Tutorials*, 2022.

[7] R. Ghafari, F. H. Kabutarkhani, and N. Mansouri, "Task scheduling algorithms for energy optimization in cloud environment: a comprehensive review," *Cluster Computing*, vol. 25, no. 2, pp. 1035–1093, 2022.

[8] G. U. Srikanth and R. Geetha, "Effectiveness review of the machine learning algorithms for scheduling in cloud environment," *Archives of Computational Methods in Engineering*, pp. 1–21, 2023.

[9] Y. N. Khalid, M. Aleem, U. Ahmed, M. A. Islam, and M. A. Iqbal, "Troodon: A machine-learning based load-balancing application scheduler for cpu–gpu system," *Journal of Parallel and Distributed Computing*, vol. 132, pp. 79–94, 2019.

[10] M. S. Ahmed, S. Biswas, M. J. Rahman, M. H. R. Alhadi, R. A. Moon, and M. M. Islam, "Machine learning for load forecasting in a green data center," in *2022 4th International Conference on Sustainable Technologies for Industry 4.0 (STI)*. IEEE, 2022, pp. 1–5.

[11] S. A. Murad, Z. R. M. Azmi, A. J. M. Muzahid, and M. Al-Imran, "Comparative study on job scheduling using priority rule and machine learning," in *2021 Emerging Technology in Computing, Communication and Electronics (ETCCE)*. IEEE, 2021, pp. 1–8.

[12] B. Pérez, J. L. Bosque, and R. Beivide, "Simplifying programming and load balancing of data parallel applications on heterogeneous systems," in *Proceedings of the 9th Annual Workshop on General Purpose Processing using Graphics Processing Unit*, 2016, pp. 42–51.

[13] M. Boyer, K. Skadron, S. Che, and N. Jayasena, "Load balancing in a changing world: dealing with heterogeneity and performance variability," in *Proceedings of the ACM International Conference on Computing Frontiers*, 2013, pp. 1–10.

[14] P. Huchant, M.-C. Counilh, and D. Barthou, "Automatic opencl task adaptation for heterogeneous architectures," in *Euro-Par 2016: Parallel Processing: 22nd International Conference on Parallel and Distributed Computing, Grenoble, France, August 24-26, 2016, Proceedings 22*. Springer, 2016, pp. 684–696.

[15] Z. Lin, C. Li, L. Tian, and B. Zhang, "A scheduling algorithm based on reinforcement learning for heterogeneous environments," *Applied Soft Computing*, vol. 130, p. 109707, 2022.

[16] F. Cheng, Y. Huang, B. Tanpure, P. Sawalani, L. Cheng, and C. Liu, "Cost-aware job scheduling for cloud instances using deep reinforcement learning," *Cluster Computing*, pp. 1–13, 2022.

[17] N. Aslam, N. Sarwar, and A. Batool, "Designing a model for improving cpu scheduling by using machine learning," *International Journal of Computer Science and Information Security*, vol. 14, no. 10, p. 201, 2016.

[18] P. K. Bal, S. K. Mohapatra, T. K. Das, K. Srinivasan, and Y.-C. Hu, "A joint resource allocation, security with efficient task scheduling in cloud computing using hybrid machine learning techniques," *Sensors*, vol. 22, no. 3, p. 1242, 2022.

[19] G. Vasiliadis, R. Tsirbas, and S. Ioannidis, "The best of many worlds: Scheduling machine learning inference on cpu-gpu integrated architectures," in *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2022, pp. 55–64.

[20] T. Helmy, S. Al-Azani, and O. Bin-Obaidellah, "A machine learning-based approach to estimate the cpu-burst time for processes in the computational grids," in *2015 3rd International Conference on Artificial Intelligence, Modelling and Simulation (AIMS)*. IEEE, 2015, pp. 3–8.

[21] J. Gao, H. Wang, and H. Shen, "Machine learning based workload prediction in cloud computing," in *2020 29th international conference on computer communications and networks (ICCCN)*. IEEE, 2020, pp. 1–9.

[22] C. Battiloro, M. Merluzzi, P. Di Lorenzo, and S. Barbarossa, "Dynamic resource optimization for adaptive federated learning empowered by reconfigurable intelligent surfaces," in *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 4083–4087.

[23] N. V. Janagoudar, D. Narayan, and M. M. Mulla, "Multi-objective scheduling using logistic regression for openstack-based cloud," *Procedia Computer Science*, vol. 171, pp. 1429–1438, 2020.

[24] Y. Shen, "Virtual resource scheduling prediction based on a support vector machine in cloud computing," in *2015 8th international symposium on computational intelligence and design (ISCID)*, vol. 1. IEEE, 2015, pp. 110–113.